Operadores de atribuição, aritméticos, relacionais e lógicos no Java

Os **operadores de atribuição**, **aritméticos**,**relacionais** e **lógicos** no **Java** são utilizados principalmente na etapa de processamento - para a construção da lógica - possibilitando realizar ações específicas sobre os dados. Adição, subtração, multiplicação, comparação são apenas alguns exemplos.

Neste documento apresentamos os principais operadores utilizados para a escrita de algoritmos em **Java**.

Guia relacionado: Guia de Referência Java

Tópicos

Operador de atribuição

Operadores aritméticos

Operadores de incremento e decremento

Operadores de igualdade

Operadores relacionais

Operadores lógicos

Precedência de operadores

Exemplo prático

Operadores de atribuição em Java

O operador de atribuição é utilizado para definir o valor inicial ou sobrescrever o valor de uma variável. Em seu uso, o operando à esquerda representa a variável para a qual desejamos atribuir o valor informado à direita.

Exemplo de uso:

```
int lado = 2;
float pi = 3.1426F;
String texto = “DevMedia”;
lado = 3;
```

Nesse exemplo iniciamos as variáveis *lado*, *pi* e *texto*, sobrescrevendo a variável *lado* em seguida.

Operadores aritméticos

Os operadores aritméticos realizam as operações fundamentais da matemática entre duas variáveis e retornam o resultado. Caso seja necessário escrever operações maiores ou mais complexas, podemos combinar esses operadores e criar expressões, o que nos permite executar todo tipo de cálculo de forma programática.

Exemplo de uso:

```
int area = 2 * 2;
```

Esse código demonstra como calcular a área de um quadrado de lado igual a 2.

Também podemos utilizar os operadores aritméticos em conjunto com o operador de atribuição, realizando, em uma mesma instrução, as ações de calcular o valor e atribuí-lo à variável.

Exemplo de uso:

```
int area = 2;
area *= 2;
```

Nota: *A segunda linha desse código é equivalente a* `area = area * 2`.

Opções de operadores aritméticos

A tabela abaixo apresenta os **operadores aritméticos** da linguagem **Java**:

+	operador de adição
-	operador subtração
*****	operador de multiplicação
/	operador de divisão
%	operador de módulo

	(ou resto da divisão)

Operadores de incremento e decremento

Os **operadores de incremento** e decremento também são bastante utilizados. Basicamente temos dois deles:`++` e `--`, os quais podem ser declarados antes ou depois da variável e incrementam ou decrementam em 1 o valor da variável.

Exemplo de uso:

```
int numero = 5;
numero++;
numero--;
//numero continuará sendo 5.
```

Quando declaramos esse operador antes da variável, o incremento é realizado antes do valor da variável ser lido para o processamento ao qual a instrução pertence. Quando declarado depois, ocorre o contrário: lê-se o valor da variável para processamento e só então o valor da variável é incrementado. Com base nisso, suponha que temos o código abaixo:

Exemplo de uso:

```
int desafioUm = 5;
System.out.println(desafioUm  += ++desafioUm );

int desafioDois = 5;
System.out.println(desafioDois  += desafioDois++);
```

Quais valores serão impressos no console? 10 e 10, 10 e 11, 11 e 10 ou 11 e 11? A resposta é 11 e 10.

No primeiro *println()*, *desafioUm* é incrementado antes de seu valor ser lido para compor a instrução de soma. Sendo assim, temos *desafioUm = 5 + 6*. Já no segundo *println()*, primeiro o valor é lido, resultando em *desafioDois = 5 + 5*. Somente após a leitura *desafioDois* é incrementado, e depois, recebe o valor da soma, tendo seu valor sobrescrito com o número 10.

Operadores de igualdade

Os operadores de igualdade verificam se o valor ou o resultado da expressão lógica à esquerda é igual (“`==`”) ou diferente (“*!=*”) ao da direita, retornando um valor booleano.

Exemplo de uso:

```
int valorA = 1;
int valorB = 2;

if(valorA == valorB){
                System.out.println(“Valores iguais”);
} else {
                System.out.println(“Valores diferentes”);
}
```

Esse código verifica se duas variáveis contêm o mesmo valor e imprime o resultado. Uma vez que as variáveis *valorA* e *valorB* possuem valores diferentes, o trecho de código presente no *else* será executado. Caso ainda não conheça as estruturas de condição, acesse: Documentação Java: if/else e o operador ternário.

Opções de operadores de igualdade

A tabela abaixo apresenta os **operadores de igualdade** do **Java**:

==	Utilizado quando desejamos verificar se uma variável é igual a outra.
!=	Utilizado quando desejamos verificar se uma variável é diferente de outra.

Nota: *Os operadores de igualdade normalmente são utilizados para comparar tipos primitivos (byte, short, int, long, float, double, boolean e char). No entanto, também podemos utilizá-los para saber se duas instâncias estão apontando para o mesmo objeto.*

Operadores relacionais

Os **operadores relacionais**, assim como os de igualdade, avaliam dois operandos. Neste caso, mais precisamente, definem se o operando à esquerda é menor, menor ou igual, maior ou maior ou igual ao da direita, retornando um valor booleano.

Exemplo de uso:

```
int valorA = 1;
int valorB = 2;

if (valorA > valorB) {
                System.out.println(“maior”);
}

if (valorA >= valorB) {
                System.out.println(“maior ou igual”);
}

if (valorA < valorB) {
                System.out.println(“menor”);
}

if (valorA <= valorB) {
                System.out.println(“menor ou igual”);
}
```

Esse código realiza uma série de comparações entre duas variáveis para determinar o que será impresso no console. Uma vez que o valor da variável `valorA` é menor que `valorB` serão impressas as mensagens “menor” e “menor ou igual”.

Opções de operadores relacionais

A tabela abaixo apresenta os operadores relacionais do Java:

>	Utilizado quando desejamos verificar se uma

	variável é maior que outra.
>=	Utilizado quando desejamos verificar se uma variável é maior ou igual a outra
<	Utilizado quando desejamos verificar se uma variável é menor que outra.
<=	Utilizado quando desejamos verificar se uma variável é menor ou igual a outra.

Operadores lógicos

Os **operadores lógicos** representam o recurso que nos permite criar expressões lógicas maiores a partir da junção de duas ou mais expressões. Para isso, aplicamos as operações

lógicas E (representado por “&&”) e OU (representado por “||”).

Exemplo de uso:

```
if((1 == (2 -1)) && (2 == (1 + 1))){
     System.out.println(“Ambas as expressões são verdadeiras”);
}
```

Uma vez que utilizamos o operador lógico *&&*, o *System.out.println* somente será executado se as duas condições declaradas no *if* forem verdadeiras.

Opções de operadores de lógicos

A tabela abaixo apresenta os operadores lógicos do Java:

&&	Utilizado quando desejamos que as duas expressões sejam verdadeiras.
||	Utilizado quando precisamos que pelo meno um das expressões seja verdadeira.

Precedência de operadores

Uma vez que os operadores aritméticos buscam reproduzir as operações matemáticas fundamentais, é natural que eles mantenham as suas regras de precedência, que podem ser manipuladas pelo programador com o uso de parênteses.

Por exemplo, a expressão 1 + 1 * 2, quando analisada pelo compilador, vai retornar o valor 3, porque a multiplicação será resolvida antes da adição. Usando parênteses, a expressão (1 + 1) * 2 retornará o valor 4, pois a adição, por estar dentro dos parênteses, será resolvida primeiro.

Exemplo de uso:

```
if ((1 != (2 -1)) || (2 == (1+1))) {
  System.out.println(“iguais”);
}
```

Nota: *Para facilitar a leitura das expressões e evitar erros de lógica, é recomendado o uso dos parênteses para separar e agrupar as condições.*

Exemplo prático

Suponha que você precisa programar um código simples para definir o salário dos funcionários de uma empresa considerando o tempo que cada um tem nessa empresa e o número de horas trabalhadas. Para tanto, podemos utilizar alguns dos operadores apresentados nessa documentação.

Exemplo de uso:

```
int quantidadeAnos = 5;
int horasTrabalhadas = 40;
int valorHora = 50;
int salario = 0;

if (quantidadeAnos <= 1) {
    salario = 1500 + (valorHora * horasTrabalhadas);
} else if ((quantidadeAnos > 1) && (quantidadeAnos < 3)) {
    salario = 2000 + (valorHora * horasTrabalhadas);
} else {
    salario = 3000 + (valorHora * horasTrabalhadas);
}
```